CONFIDENTIAL — TECHNICAL BRIEFING

Unfilter Story

System Analysis & Architecture Review

Startup-News Intelligence Platform

Prepared for: Chief Technology Officer
Repository: `unfilter-story-1`
Date: 29 May 2026

1. What This Product Is

Unfilter Story is an **Indian startup-news intelligence platform**. It has two faces:

- **A public news site** — readers see curated articles plus a live "Startup News Feed."
- **An internal CMS / intelligence console** — the editorial team discovers, classifies, curates, AI-rewrites, and publishes startup news.

The core differentiator is the **Discovery Engine**: it continuously ingests ~22 RSS sources (Inc42, YourStory, Entrackr, Economic Times, Google News queries, etc.), then runs a **keyword-based classification engine** that tags each story by *signal type* (Funding, Acquisition, Layoffs, Shutdown, ...) and *industry vertical* (Fintech, AI/ML, HealthTech, ...), while filtering out stock-market and corporate noise.

2. Architecture — Three Independent Apps

Layer	Tech	Location
Backend / API	Fastify 5, Prisma 6, rss-parser, ESM Node	`api-backend/`
Database	**SQLite** (`prisma/dev.db`)	`api-backend/prisma/`
Admin CMS	React 19, Vite 7, React Router 7, TipTap editor, Tailwind 4, dnd-kit	`cms-admin/`
Public site	Astro 6 (SSR via Node adapter), React islands, Tailwind 4	`public-site/`
AI	Google Gemini SDK + Bytez.js (multi-model proxy)	backend
Infra (defined, unused)	Postgres 16 + Redis 7 via Docker	`docker-compose.yml`

3. The Backend — Feature by Feature

The backend is a **single 1,562-line** `index.js` (monolithic — flagged below). It exposes:

Public API (`/v1/*`)

- `GET /v1/articles` — published articles (latest 20)

- `GET /v1/articles/:slug` — single article + auto-increments `viewCount`
- `POST /v1/contact` — contact-form submissions

CMS API (`/cms/v1/*`) — full CRUD for:

- Articles (auto-slug, reading-time calc, tag find-or-create)
- Categories, Tags, Media, Navigation (with drag-reorder), Users, Site Settings, Contact messages

AI Studio (`/cms/v1/ai/transform`)

Rewrites / transforms editorial text. Routes "gemini" → `google/gemma-2-27b-it` and "gpt" → `openai/gpt-4o-mini`, **both via Bytez** (a code comment notes this bypasses Gemini free-tier rate limits).

Discovery / RSS Engine (`/cms/v1/rss/*`) — the crown jewel

- `GET /rss/fetch` — paginated, filterable feed (by source, category, date range, bookmarks). Auto-seeds 22 sources on first call.
- `POST /rss/trigger-sync` — manual sync (incremental or deep)
- `GET /rss/sync-status` — live progress (sources done, signals added, elapsed ms)
- `POST /rss/bookmark/:id` — toggle bookmark

The Discovery Sync — Two-Tier Architecture

1. **Deep Sync** (`runDeepSync`) — historical backfill. Paginates up to 100 pages per source, 6-month lookback. Runs once on first boot (empty DB).
2. **Incremental Sync** (`runIncrementalSync`) — fast top-of-feed scan (page 1 only), stops early when it hits known items. Runs on boot + **every 30 minutes**.
3. **Data Retention** (`runDataRetentionCleanup`) — daily; prunes signals older than ~6 months **unless bookmarked**.

The Classification Engine (the real IP)

For every item it:

1. **Noise filter** — discards Sensex/Nifty/Ambani/Adani/crude-oil/geopolitics headlines.
2. **Industry tagging** — 25 verticals (Fintech, AI/ML, SpaceTech…), keyword regex with word boundaries.
3. **Signal tagging** — 22 prioritized signal types (Funding = priority 1 … Innovation = 21), with "strong" vs "supporting" keyword tiers and a confidence threshold. Headline matches outweigh body matches; capped to the single top signal + all matched industries.

⚠ **Note:** This logic is **duplicated** — once in `runDeepSync` and again (slightly trimmed) in `runIncrementalSync`. The same ~250-line keyword dictionary is maintained in two places. A prime refactor target.

4. Data Model (Prisma)

15 models: `CmsUser`, `Category` (self-nesting), `Tag`, `Article` (rich: SEO fields, access levels, breaking-news flag, scheduling), `ArticleTag`, `ArticleAuthor`, `ArticleVersion` (versioning), `Media`, `AuditLog`, `Redirect`, `Navigation` (hierarchical), `RssSource`, `DiscoveryCache` (the ingested signals), `SiteSetting` (key-value), `ContactMessage`.

The schema is well-designed for a real CMS — versioning, audit logs, multi-author, and redirects are all modeled (though not all wired up yet).

5. The Admin CMS (React)

13 pages: Dashboard, Articles, ArticleEditor (TipTap rich editor with tables, images, YouTube), Media, Taxonomy, Navigation (drag-and-drop), **Discovery** (the intelligence console), **AIStudio**, Architecture, Users, Settings, ContactMessages.

6. The Public Site (Astro SSR)

Pages: home, `article/[slug]`, `startup-news` (live feed), about, contact, privacy, terms. Header/Footer pull branding from `/cms/v1/settings` at render time.

7. How to Set Up & Run

The DB is **SQLite** in code, so Docker/Postgres is **not** strictly required to run it today.

Backend

```
cd api-backend
npm install
npx prisma generate          # generate client
npx prisma migrate dev       # or: npx prisma db push  (creates dev.db)
# create .env  →  copy .env.example (see DB note below)
node index.js                # port 3000; auto deep-sync if DB empty
```

Admin CMS

```
cd cms-admin
npm install
npm run dev                  # Vite dev server (usually :5173)
```

Public site

```
cd public-site
npm install
npm run dev                  # Astro dev server (usually :4321)
```

First backend boot with an empty DB triggers a **full historical deep sync** (slow, hits all 22 feeds) — let it run in the background; the feed populates progressively.

8. CTO-Level Risks & Recommendations

#	Issue	Severity	Recommendation
1	**DB mismatch**: code uses SQLite, but `.env.example` + `docker-compose.yml` declare Postgres. Confusing & not production-ready.	**HIGH**	Pick one. For production, migrate `schema.prisma` to `postgresql` and use the Docker Postgres. SQLite won't handle concurrent sync + web traffic.
2	**No authentication anywhere.** All `/cms/v1/*` routes are open; `CORS origin: '*'`; users created with `passwordHash: 'placeholder'`. JWT is a dependency but unused.	**CRITICAL**	Add auth middleware + JWT login before any deployment. Currently anyone can delete all articles.
3	**Hardcoded** `http://localhost:3000` in ~40 places across both frontends.	**MEDIUM**	Extract to an env var (`VITE_API_URL` / `PUBLIC_API_URL`). Blocks deployment as-is.
4	**1,562-line monolith** `index.js` with the classification dictionary **duplicated**.	**MEDIUM**	Split into routes/services; extract the taxonomy into one shared module.
5	**In-memory sync state** + `setInterval` schedulers. State lost on restart; won't scale horizontally. Redis is provisioned but unused.	**LOW–MED**	Move schedulers/state to a job queue (BullMQ + the existing Redis).
6	**Silent error swallowing** — many `.catch(() => {})` in the sync engine.	**LOW**	Log failures with context.
7	~20 loose `test_*.js` / utility scripts in `api-backend/` root (model probes, data dumps).	**CLEANUP**	Move to a `scripts/` folder or remove.

TL;DR

A **3-tier startup-news intelligence platform**: Astro public site + React admin CMS + Fastify/Prisma backend. The standout is a **two-tier RSS discovery engine** that ingests 22 Indian startup sources every 30 minutes and auto-classifies stories into 22 signal types × 25 industries via a keyword engine. The CMS is genuinely full-featured (rich editor, AI rewriting, versioning, media, taxonomy).

It is a strong prototype but not production-ready. The three blockers are: **no auth**, **SQLite-vs-Postgres confusion**, and **hardcoded localhost URLs**.